

Bases de données - Partie 2

Plan de cours

- Concevoir une **API REST** avec **Express.js** en **MySQL**
- Implémenter un **accès aux données découplé** (DAO / IDAO)
- Comprendre l'**abstraction de la couche base de données**
- Connecter une application **Angular** à une API backend
- Mettre en pratique via un **projet complet**

Stack technique

- Backend : Node.js, Express.js
- Bases de données : MySQL & MongoDB
- Frontend : Angular
- Architecture : Couplage faible, séparation des responsabilités

Prérequis

- JavaScript / TypeScript
- Bases de HTTP et REST
- Notions Angular et Node.js
- MongoDB et SQL (niveau introductif)

1. Express & ORM MySQL

Mise en place côté Backend



Rappels : Express.js

Express est un **framework minimaliste** pour Node.js permettant de :

- Créer une API REST
- Gérer le routage HTTP
- Structurer un backend modulaire
- Interagir avec une base de données

Pourquoi utiliser un ORM ?

Un ORM (Object Relational Mapping) permet de :

- Manipuler la base de données via des **objets JavaScript**
- Éviter l'écriture de SQL brut
- Améliorer la **maintenabilité** du code
- Faciliter l'évolution du schéma
- Limiter les erreurs SQL

ORM choisi : Sequelize

Dans ce cours, nous utilisons **Sequelize** :

- Compatible MySQL, PostgreSQL, SQLite
- Basé sur des **modèles**
- Supporte les migrations
- Très utilisé avec Express

Installation des dépendances

Dans le projet Express :

- Initialiser le projet
- Installer Express et Sequelize
- Installer le driver MySQL

Commandes :

```
npm init -y  
npm install express sequelize mysql2  
npm install --save-dev nodemon
```

Structure de projet recommandée

```
mon-projet/  
| — src/  
|   | — config/  
|   |   | — database.js  
|   | — models/  
|   | — services/  
|   | — routes/  
|   | — app.js  
| — package.json
```

Configuration de la base MySQL

Créer le fichier database.js :

- Paramètres de connexion
- Instance Sequelize
- Export de la connexion

Exemple :

```
const { Sequelize } = require('sequelize')

const sequelize = new Sequelize(
  'database_name',
  'user',
  'password',
  {
    host: 'localhost',
    dialect: 'mysql'
  }
)

module.exports = sequelize
```

Connexion à la base de données

Dans app.js :

- Importer Sequelize
- Tester la connexion au démarrage

```
sequelize.authenticate()  
  .then(() => console.log('Connexion MySQL OK'))  
  .catch(err => console.error('Erreur MySQL', err))
```

Définition d'un modèle Sequelize

Un modèle représente une **table SQL**.

Exemple : User

- Champs
- Types
- Contraintes

id : INT

email : STRING

password : STRING

Exemple de modèle User

```
const { DataTypes } = require('sequelize')
const sequelize = require('../config/database')

const User = sequelize.define('User', {
  id: {
    type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true
  },
  email: {
    type: DataTypes.STRING, allowNull: false,
  }
})

module.exports = User
```

Synchronisation avec la base

Créer automatiquement les tables :

```
sequelize.sync()  
  .then(() => console.log('Tables synchronisées'))  
  .catch(err => console.error(err))
```

PS: À utiliser uniquement en développement

Utilisation du modèle (CRUD)

Créer un utilisateur :

```
User.create({  
  email: 'test@mail.com',  
  password: '1234'  
})
```

Lire tous les utilisateurs :

```
User.findAll()
```

Séparation des responsabilités

Bonne pratique :

- **Routes** : HTTP
- **Services** : logique métier
- **Models** : accès données

Objectif : **code maintenable et évolutif**

Conclusion

- Express opérationnel
- Connexion MySQL fonctionnelle
- ORM configuré
- Modèles Sequelize créés
- CRUD de base opérationnel

